

Project Proposal

The convex hull of an input set of points is the convex polygon that contains all the input points. Also, the vertices of this convex polygon are a subset of the input points. Finding convex hulls has far-reaching applications in many areas of research, from thermodynamics and quantum physics to statistics and economics.

For this project I will implement Jarvis's, Chan's, and Andrew's convex hull algorithms. I will implement these algorithms in C++ and I will handle as many edge cases as possible. I will evaluate the wall time of each of these algorithms with varying inputs. Then I will implement and analyze the performance of optimizations to these algorithms.

Let n denote the total number of points and h denote the number of points on the convex hull. The best-case input for Jarvis's algorithm is when $h \ll n$. For example, a best-case input for Jarvis's algorithm would be a cluster of points in the center and then three points forming a triangle which contains the cluster of points. Jarvis's algorithm has time complexity $O(nh)$. Chan's algorithm has time complexity $O(n \log h)$. Andrew's algorithm has time complexity $O(n \log n)$. The best-case input for Andrew's algorithm is when $n = h$, such as a circle. Theoretically, Chan's algorithm is asymptotically as fast as Jarvis's and Andrew's. I will test the performance of these algorithms with different inputs and see which algorithm performs best for each combination of input shape and input size.

While Jarvis's and Andrew's algorithms are implemented in libraries such as CGAL, Chan's is much less common. Through implementing these algorithms, I might learn the tradeoffs between Jarvis's, Andrew's, and Chan's algorithms which cause Chan's algorithm to be seldom implemented.

An analysis of the runtimes of convex hull algorithms has already been done before in "Convex Hulls: Comparing Theoretical and Practical Runtimes" by Zach Havens and Danika Passler Bates. They found that Chan's algorithm had the worst performance, despite being theoretically the best. My work will differ from theirs because while they used Python, I will use C++, possibly eliminating some overhead

which could have muddled their results. Additionally, they added several areas of future work. From their future work section I will implement a couple of optimizations for Chan's algorithm and a preprocessing step that could potentially speed up all the algorithms. The first potential optimization of Chan's algorithm was to reuse existing sub-hulls and merge them instead of recalculating sub-hulls. Another optimization of Chan's algorithm was using a heuristic to set an initial number of sub-hulls, based on the input size or knowledge of the point distribution shape. Lastly, a preprocessing step which might improve the performance of all the convex hull algorithms is to eliminate a set of points from consideration for the convex hull. This is done by picking four points with the least x, greatest x, least y, and greatest y. These points form the corners of a convex quadrilateral and we can safely remove from consideration all points inside this quadrilateral, except the corner points. I will see which of these optimizations is most beneficial. Also, these optimizations could help explain why Chan's algorithm is so slow.